

The Dengue Exercise

Map a search signal onto disease activity, with an agent

Shihao Yang • Candice Djorno

July 20, 2026

SISMID 2026 • Day 1, 11:00

An hour ago you **scraped** dengue search interest for Mexico. Now the real question:

Does a search signal actually track the disease?

We will build a tiny **Google Dengue Trends**: fit search onto reported cases on the early years, then predict the later years and see how well it holds up.

- Same two lanes as this morning: **Lane A** prompt an agent (Codex, Claude Code, Antigravity CLI) with `notebooks/dengue_exercise.ipynb`; **Lane B** the pre-filled `notebooks/dengue_exercise_soln.ipynb`.
- You will paste *one prompt per step* and keep the judgment.

Intro to Google Dengue Trends

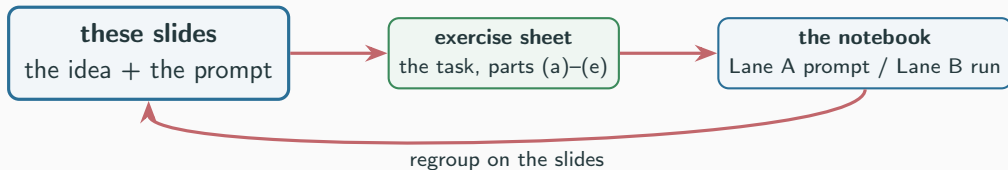
Doing it with an agent

Stretch: dynamic training and Lasso

Wrap-up

How this hour runs (for the instructor)

You move among three things; the slides cue every hand-off.



- **Hand out now:** `exercise/dengue-exercise-2026.pdf` (the task, a–e).
- **Open now:** `notebooks/dengue_exercise.ipynb` (Lane A) or `..._soln.ipynb` (Lane B).
- Follow the ► **to the notebook** and ◀ **back to slides** cues at the corners.

Intro to Google Dengue Trends

The idea behind Google Dengue Trends

When dengue rises, people search: *dengue, sintomas de dengue, mosquito*.

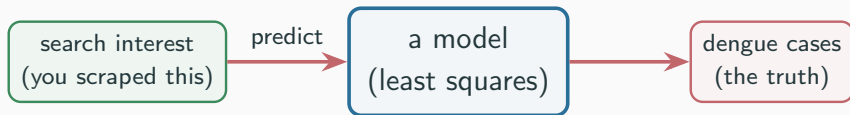
- Aggregate search volume moves **with** disease activity, and it is available **cheaply and in near-real-time**, ahead of official case counts.
- This is the digital-epidemiology origin story: **Google Flu Trends** (2009) nowcast influenza from searches; the same recipe was tried for dengue in tropical countries.

The whole field starts from one plot: a search curve and a disease curve that rise and fall together.

The data

MX_Dengue_trends.csv (Mexico, monthly, 2004–2011):

- Dengue CDC — reported dengue **cases** (Mexican Ministry of Health). This is the **ground truth** we try to match.
- dengue, sintomas de dengue, mosquito, dengue sintomas — Google **search** interest, the same kind of series you scraped this morning.



Exercise sheet: [exercise/dengue-exercise-2026.pdf](#).

The task in one line

Goal

Fit **search** → **cases** on 2004–2006, then use that line to **predict** cases for 2007–2011 from search alone, and judge how well a cheap search signal stands in for slow case reports.

Five parts (from the exercise sheet):

1. plot cases over time,
2. fit a least-squares line on the training years,
3. show the fit on the training years,
4. predict and check the validation years,
5. discuss: how would you improve it?

► Hand out the sheet, open the notebook; the next slides are the prompts, one per step.

Doing it with an agent

Agent-first, one prompt per step

You will not hand-write pandas. You describe each step and its check; the agent writes and runs the code.



Lane B has the same steps pre-run, so a stuck neighbor is never blocked.

Step 1: visualize (look before you model)

Prompt (paste this)

```
Load MX_Dengue_trends.csv. "Dengue CDC" is monthly dengue cases in Mexico; "dengue" is Google search interest. Plot the cases over time. Then overlay the "dengue" search series on a second y-axis, and print the correlation between the two columns.
```

- You are checking three things: do they **move together**? is there a **lead or lag**? any **gaps or spikes** that will fool a model?
- If the two curves clearly track each other, a straight line already has a chance.

Step 2: least squares, the honest baseline

Let y_t be dengue cases and x_t the dengue search volume. Fit on 2004–2006:

$$y_t = a + b x_t + \varepsilon_t, \quad (\hat{a}, \hat{b}) = \arg \min_{a,b} \sum_t (y_t - a - b x_t)^2.$$

Prompt (paste this)

```
Using only the first 36 months (2004-2006) as training,  
fit ordinary least squares: Dengue CDC ~ dengue. Report the  
intercept and slope, the in-sample R^2, and overlay the  
fitted line on a scatter of cases vs search.
```

One search term, one slope. This is Google Dengue Trends stripped to its core.

Step 3: predict, then judge out of sample

In-sample fit always looks good. What matters is the years the line has *not* seen.

Prompt (paste this)

```
Using the intercept and slope from training, predict dengue
cases for 2007-2011 from the "dengue" search column alone.
Plot predicted vs actual cases over time, and report the
out-of-sample correlation and RMSE. Do NOT refit on 2007+.
```

- Report out-of-sample error, not just R^2 . That gap is the lesson.
- **Your check:** the last training point is 2006, and nothing after 2006 touched the fit. Agents sometimes leak the future; catch it.

◀ Once everyone has run the three steps, back to the slides to check the work.

Verify what the agent produced

Before you trust a number, run one plot and one question:

- **Split is honest:** training ends 2006, prediction uses only search.
- **Scales and axes:** predicted cases in a plausible range, dates aligned.
- **Face validity:** do predicted peaks land in the right seasons?
- **Reproducible:** rerun; the fit should not wander.

The habit

The agent is fast, not infallible. Every output earns one sanity check before it counts. That is the skill this exercise builds.

Step 4: a good fit is not a correct model

- A single search term can track a disease for a while because both track the **season**, then break when behavior changes.
- **Google Flu Trends** is the famous cautionary tale: it beat lagged CDC data for years, then badly overshot as media coverage and the search algorithm shifted.

Identifiability

Fitting well does not prove the mechanism is right. Validate out of sample, re-check every season, and stay humble about what a correlation means.

This is exactly why tomorrow's ARGO adds the disease's own history and refits as it goes.

Stretch: dynamic training and Lasso

If you finished early: a dynamic model with Lasso

Stretch task on the sarahbellum/Zika-tutorial data.

Prompt (paste this)

```
Using the Zika tutorial data, fit the search-to-cases model  
with DYNAMIC training: refit on a rolling 2-year window  
instead of once. With several search terms, use Lasso (L1)  
so useless terms drop out. Compare its out-of-sample error  
to the single-term static line from before.
```

$$\min_{a, \beta} \sum_t \left(y_t - a - \sum_j \beta_j x_{j,t} \right)^2 + \lambda \sum_j |\beta_j|.$$

► Stretch runs in the notebook too; autoregression itself waits for ARGO on Day 2.

Why these two upgrades

Dynamic training

- refit on a rolling window
- adapts as the search-disease link drifts
- the fix Google Flu Trends lacked

Lasso (ℓ_1)

- many candidate search terms in
- only the useful ones survive
- exactly what you want from a scraped pile of queries

Dynamic training plus term selection is the on-ramp to ARGO on Day 2.

Wrap-up

What you built

- A working **mini Google Dengue Trends**: a search signal mapped onto dengue activity in Mexico, fit on the past and **honestly validated** on the future.
- You did it by **describing steps to an agent** and checking its work, not by memorizing pandas.
- You saw the field's core warning first-hand: a great fit can still be a poor forecast.

The same loop (visualize, fit, predict, judge) is every model you will build this week.

Looking ahead: from this line to ARGO

- **This afternoon:** Google Trends in depth, why the same pull returns different numbers, and how preprocessing restores its forecasting power.
- **Day 2:** turn this one-term line into **ARGO**: add the disease's own recent history (autoregression), many selected terms, and dynamic training.

You now have the baseline the rest of the course improves on.

Questions?

Sheet: `exercise/dengue-exercise-2026.pdf` • **Notebook:**
`notebooks/dengue_exercise(_soln).ipynb`